Imperial College
London

Lecture 10

Delay in Digital VLSI Circuits

Prof Peter YK Cheung

Department of Electrical & Electronic Engineering

URL: www.ee.ic.ac.uk/pcheung/teaching/EE4-VLSI/
E-mail: p.cheung@imperial.ac.uk

In this lecture, we will consider what causes delay in Digital VLSI circuit and how to optimize such delay for better performance. We will also examine how many stages of logic would be best given a certain capacitance load.

This lecture is heavily relying on the method of Logic Effort developed by Dr Ivan Sutherland while he was taking a sabbatical at Imperial College in the early 90s!

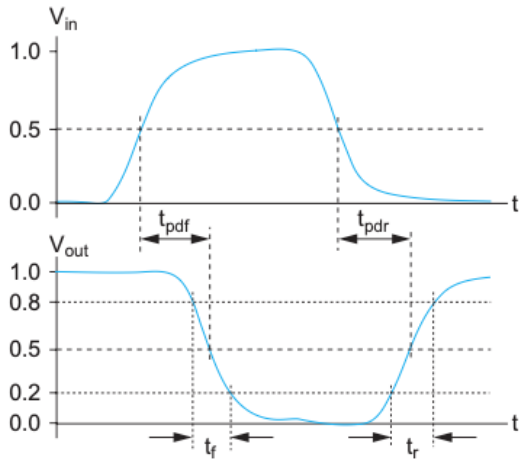
Key delay parameters

Propagation delay, t_{pd}
maximum time from the input crossing 50% to the output crossing 50%

Rise time, t_r
time for a waveform to rise from 20% to 80% of its steady-state value

Fall time, t_f
time for a waveform to fall from 80% to 20% of its steady-state value

Edge rate, t_{rf}
Average of rise and fall times, $(t_r+t_f)/2$



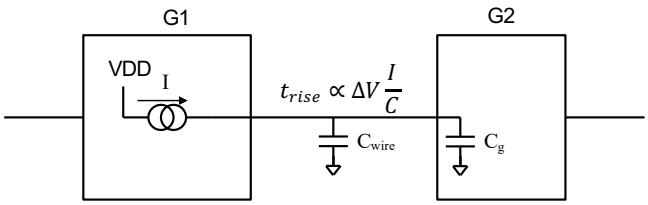
W&H p141

You should be very familiar with these definition of delay from earlier years.

What causes delay in digital CMOS?

- **Nodes** - the wires between gates
- **Driver** - the gate that charges and discharges a node
- **Delay** – worst case propagation dela

- Delay mostly caused by two factors:
  1. Gate capacitance C_g
  2. Interconnection capacitance C_{wire}
- Propagation delay of G1 $t_{pd} \approx$ time to charge/discharge its output



In digital CMOS circuits, the main cause of delay is the time it takes to change node voltages when charging and discharging capacitors. For circuits that are close together, the main source of capacitance is the gate capacitors of gate inputs. For large circuits, the wring capacitances become increasingly important.

A simplified model of the gate delay is that of a current source I driving a capacitor load as shown above. An alternative model is that of a capacitor being charged or discharged through a source resistor R.

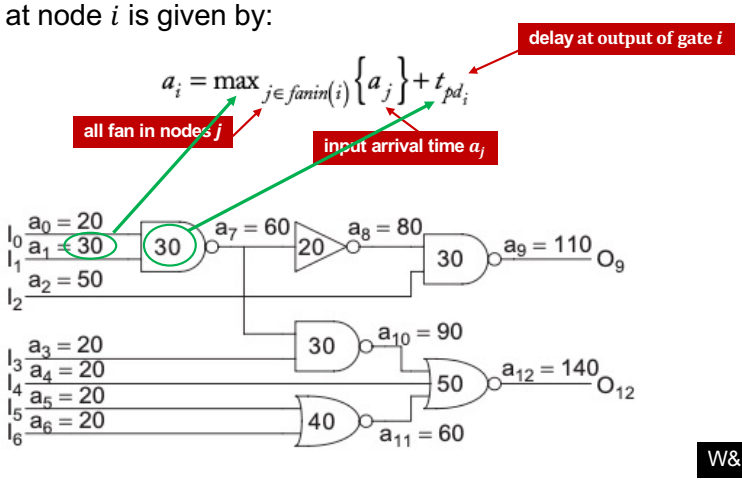
In other words, the main cause of digital circuit delay is the rise and fall time at the particular node. This delay is proportional to three factors I (or R with fixed VDD/VSS), C and ΔV.

To improve performance and decrease delay, one could:
1. Reduce power supply voltage (hence lower ΔV).
2. Reduce C by using smallest possible width and length for transistors. (That's why in digital, minimum L is always used).
3. Increase I, the driving current by increasing W of the transistor driving the node.

However, 3) is in contradiction to 2). Increasing the width of gate G1 reduces the delay between G1 and G2, but increase the input delay to G1 because the driver of G1 now has a larger capacitor to charge and discharge.

Arrival time

- Timing Analyzer computes arrival times in circuit nodes in module.
- User specifies arrival time of signals at inputs.
- User specifies required arrival time at outputs.
- Arrival time a_i at node i is given by:



W&H p142

A **timing analyzer** computes the **arrival times** at each node. , i.e., the latest time at which each node.

The arrival time at an internal node of the top left gate at output a7 is the sum of the latest (or maximum) arrival time at its inputs (a2 in this case) + the delay of this gate (which is 30 ps).

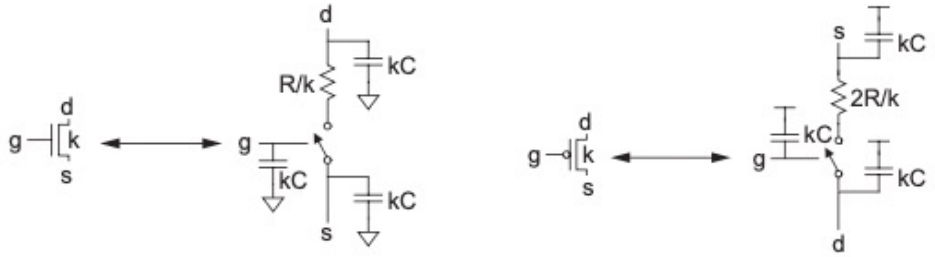
The timing analyzer computes the arrival times at each node and checks that the outputs arrive by their required time.

Slack is the difference between the required and arrival times. It is the margin in delay specification.

Positive slack means that the circuit meets timing. **Negative slack** means that the circuit is not fast enough. If the outputs are all required at 200 ps, the circuit has 60 ps of slack.

RC model of nMOS and pMOS transistors

- A unit transistor is an nMOS transistor in a minimum size inverter.
- R is the resistance, and C is gate capacitance of a unit transistor.
- nMOS transistor k x width of unit transistor has channel resistance = R/k.
- Its gate, drain and source capacitance is kC.
- pMOS mobility is around ½ that of nMOS, therefore, to have equal rise and fall times, pMOS transistor is twice as wide as nMOS transistor.
- Hence the equivalent delay model for nMOS and pMOS transistors are:



W&H p146-147

In most CMOS processes, pMOS has lower mobility than nMOS transistors. For TSMC 65nm process, the ratio μ_n to μ_p is around 2.0, the same as that used in Weste&Harris.

Here R is the unit channel resistance of a minimum size transistor and C the gate capacitance of a unit transistor.

Recall L3S6, the IV equation for an nMOS transistor in saturation (i.e. fully ON) is:

$$I_{ds} = \frac{1}{2}\mu_n C_{ox}\frac{W}{L} V_{GT}^2$$

The channel resistance is defined as:

$$R_{ch} = \frac{\delta V_{GT}}{\delta I_{ds}} = \frac{1}{\mu_n C_{ox}\frac{W}{L} V_{GT}}$$

In our process, we always use minimum L ($0.06\mu m$). Therefore, the channel resistance x width = constant:

$$R_{ch}W = \frac{\delta V_{GT}}{\delta I_{ds}} = \frac{1}{\frac{\mu_n C_{ox}}{L} V_{GT}}\ ohm.\mu m$$

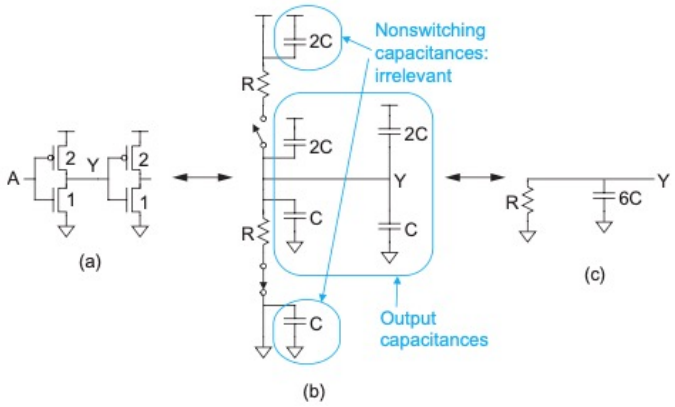
For our process, this constant is round 2.45 kΩ µm.

To simplify delay calculation, we define a unit nMOS transistor as one with minimum-size inverter in a standard cell. The effective channel resistance is *R*. Therefore if the transistor is k x minimum width, channel resistance is R/k. The capacitances at the transistor terminals are kC.

Since pMOS has ½ the mobility of nMOS transistors, to achieve roughly equal rise and fall time at the inverter output, pMOS transistor width is usually double that of nMOS transistor.

RC model of an inverter

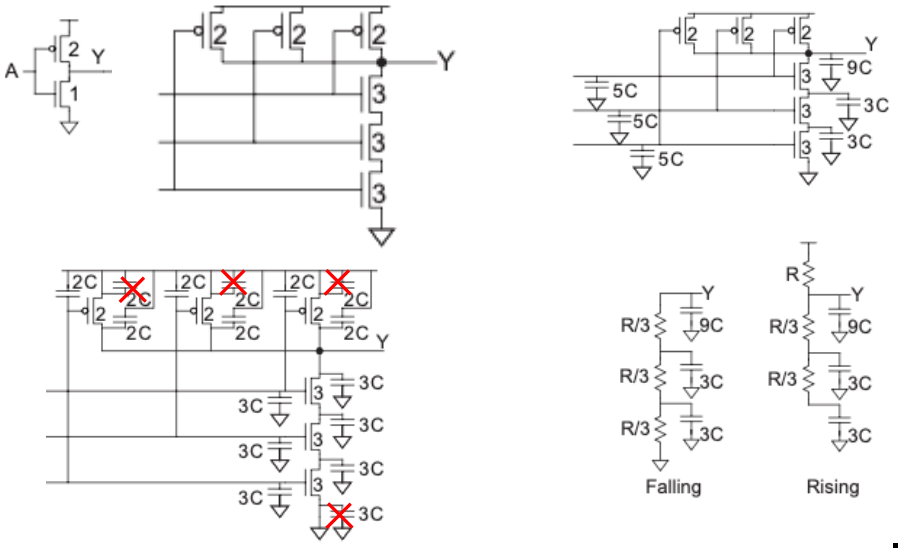
- Inverter with fanout of 1 (driving one unit inverter).
- Inverter has unit size (i.e. minimum).
- Input $0 \rightarrow 1$, the equivalent circuit is shown in b).
- Therefore, the RC model is output node driven by R and 6C.



W&H p147-148

Consider two minimum size inverter connected in series as shown. The RC model is simplified to R drive 6C. If the inverter is driver TWO identical inverter, the output capacitance will be 9C.

RC model of a 3-input NAND gate

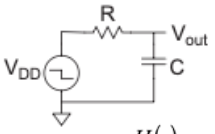


W&H p148

Consider two minimum size inverter connected in series as shown. The RC model is simplified to R drive 6C. If the inverter is driver TWO identical inverter, the output capacitance will be 9C.

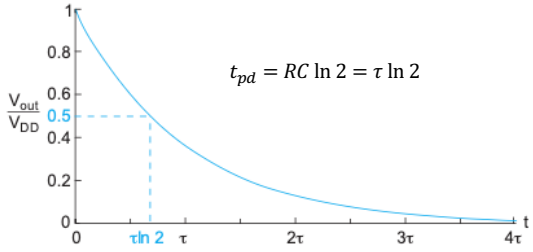
Gate delay estimation using RC model

1st order model



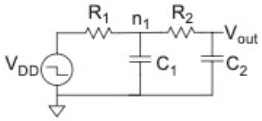
$$V_{out}(t) = V_{DD}e^{-t/\tau}$$

$$H(s) = \frac{1}{1+sRC}$$

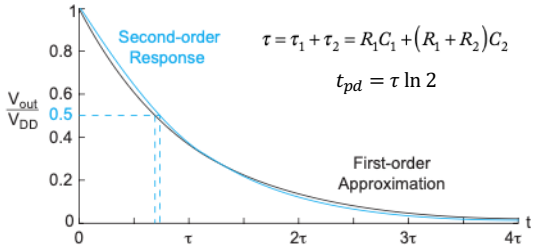


$$t_{pd} = RC\ln 2 = \tau \ln 2$$

2nd order model



$$H(s) = \frac{1}{1+s\left[R_1C_1 + (R_1+R_2)C_2\right] + s^2R_1C_1R_2C_2}$$



$$\tau = \tau_1 + \tau_2 = R_1C_1 + (R_1+R_2)C_2$$

$$t_{pd} = \tau \ln 2$$

W&H p148-149

Once the RC equivalent circuit is know, we can calculate the gate delay for a simple inverter as shown above.

One stage RC circuit has a transfer function H(s) as shown above. This is a first-order system. For a falling output voltage, solve the differential equation and we obtain the familiar exponential fall equation as shown. The fall time is the time it takes to reach 0.2 of VDD. In this case, it is time constant RC x ln 2.

What if there are two stages of RC, one driving another?

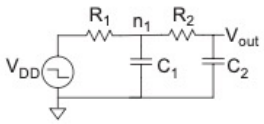
This is a second-order model as shown and the transfer function becomes much more complicate. Fortunately, this can be simplified to a first-order system with time constant as shown. The graph shows that the difference between the first-order approximation using the simplified model is very close to the exact solution.

Elmore Delay Model for RC tree

- Real circuit has many stages of RC (tree of RC circuits).
- Elmore delay model estimate delay at node *i to be*:

$$t_{pd} = \sum_i R_{is} C_i$$

1. From source VDD to Vout, there is a leave node n1 and output node Vout.
2. For each node, the delay is the time constant given by the resistance from source (VDD) to node, multiply by node capacitance. For n1, $t_{pd_{n1}} = R_1 C_1$. For Vout, $t_{pd_{V_{out}}} = (R_1 + R_2) C_2$.
3. Total delay is the sum of these two.



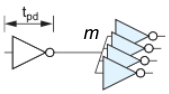
W&H p150-153

In general, most circuits of interest can be represented as an RC tree, i.e., an RC circuit with no loops. The root of the tree is the voltage source, and the leaves are the capacitors at the ends of the branches. The Elmore delay model estimates the delay from a source switching to one of the leaf nodes changing as the sum over each node i of the capacitance Ci on the node, multiplied by the total effective resistance on the shared path.

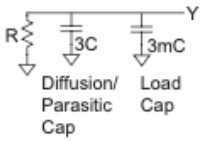
The example above explain application of the Elmore delay model to the second-order RC circuit.

Examples of using Elmore Delay Model

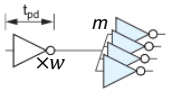
- Estimate t_{pd} for a unit inverter driving m identical unit inverters.



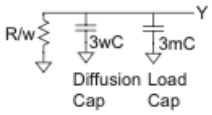
  - Total inverter input cap = $3mC$
  - Parasitic output capacitance = $3C$
  - $t_{pd} = 3(m+1)RC$



- Do the same, but now the driver inverter transistor with is w times unit size.



  - Total inverter input cap = $3mC$
  - Parasitic output capacitance = $3wC$
  - Effective resistance = R/w
  - Define fanout $h = load\ cap\ \div\ input\ cap = \frac{m}{w}$
  - $t_{pd} = \left((3w+3m)C\right)\frac{R}{W} = (3+3\mathrm{h})RC$



- If a unit transistor has R= 10 kΩ and C = 0.1 fF, compute the delay, in picoseconds, of the inverter driving four identical inverter.
  - $RC = 10k\Omega \times 0.1fF = 1ps$
  - $h = 4,$ hence $t_{pd} = (3+3\times4)\times1ps = 15ps.$

W&H p151

Here are three examples of application of the Elmore delay model.

These three examples demonstrate the following general points:

1. Delay increases linearly with output load (fanout). (Example 1)
2. Delay can be reduced by increasing the current capability of the driver. (Example 2.). However, increasing driver output current also increase the input capacitance of the driver.
3. Even 65nm process (20 years old) is FAST! The delay of a fanout_4 inverter is only 15ps.

Linear Delay Model

- RC model → linear function of gate's fanout.
- Simplify delay analysis further using normalized delay in units of τ.
- $\tau \approx 3ps$ for 65nm process .
- Linear model:

delay d = gate's effort delay f + parasitic delay p

gate's delay effort f = logical effort g × fanout h

- The complexity of the delay model is represented by the **logical effort g**
- An inverter (simplest gate) is defined to have a logic effort $g = 1$.
- More complex gate has higher logic effort. For example, a 3-input NAND gate has an effort of 5/3.
- The fanout h is called the **electrical effort h**, and is defined as:

$$h = \frac{C_{out}}{C_{in}}$$

W&H p155

The Elmore model led to the idea that delay propagation in digital VLSI could be further simplified using the so-called "linear delay model".

The idea is that a gate's delay is made up of two components:

1. The delay in the gate as it perform logical function, propagating signals from input to output. We call this logic effort delay g.
2. The delay caused by parasitic capacitors at the output node. We call this parasitic delay p.

Logical effort (or just effort) delay f depends on the logic circuit and function, and on its fanout (i.e. capacitance it drives). Parasitic delay p is only associated with the gate circuit and is not dependent on fanout.

For a given gate, we can now characterise how hard it is to drive the input of the gate to achieve the logic function. We then normalise this parameter using inverter as the reference. The detail definition of logical effort is given later.

Inverter, being a reference, has a logical effort of 1. A more complex gate, say 3-input NAND gate has a logical effort of 5/3. This is intuitively correct – a 3-input NAND gate is more complex than an inverter is therefore "costing" more in delay.

For a given logic gate function and circuit, logical effort is a constant, like the gain of an amplifier. It is independent of what load it drives.

To capture the loading effect on a gate, we define the electrical effort h.

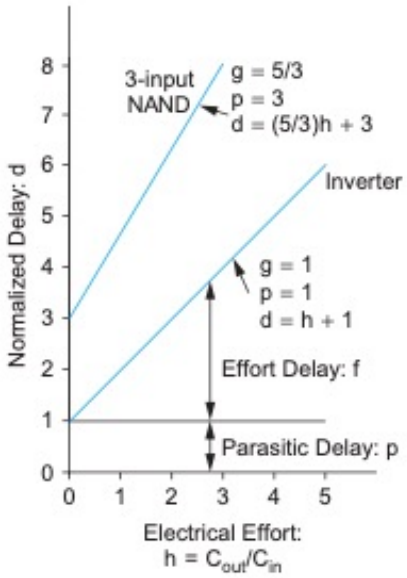
h is defined as the ratio of output capacitance of the load to the input capacitance of the gate. For the fanout_4 circuit, $h = 4$.

Delay vs Electrical Effort (fanout load)

- Model assumes linear relationship between delay and fanout load (electrical effort).
- For inverter, the plot has a gradient of 1.
- Inverter used as reference.
- The logical effort is the gradient (similar to gain of an amplifier).
- The intercept on the y-axis is the parasitic delay (no load).

- The upper plot is the same characteristic for a 3-input NAND gate.
- The logic effort g is 5/3, and that is the gradient of the line.
- The delay is:

$$d = g \times h + p = \left(\frac{5}{3}\right) h + 3.$$



W&H p155

Shown on the right is a plot of delay d normalized to the delay is the circuit was driven by an inverter, versus the electrical effort h. (h is measure on the fanout loading.)

As can be seen, the inverter is a straight line with a gradient of 1. Delay increases linearly with electrical effort. The rate of increase in delay relative to fanout is: everything we add another identical inverter at the output, the delay increase by one unit delay.

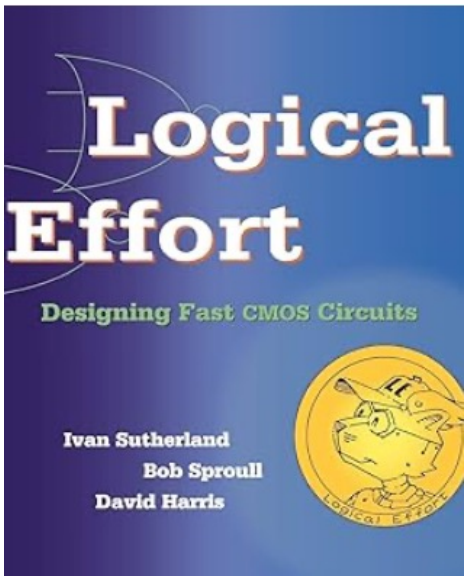
Even when there is no load (i.e. $h = 0$), there is still some delay due to parasitic capacitance. This is shown as the intercept of the line with the y-axis. In the case of an inverter, the parasitic delay $p = 1$.

The key equation here is therefore:

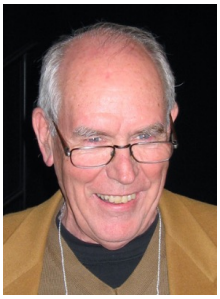
$$d = g \times h + p$$

This is a linear equation similar to that found in a linear amplifier with gain A and offset k. The output of the amplifier is = A x input + k.

Method of Logic Effort



- Introduced by Ivan Sutherland during his sabbatical at Imperial College in early 1990's.
- A simple "back of envelop" calculation of delay.
- Optimize sizes of transistors for fast operation without using software tools.



The idea of using such a simple linear model to calculate delay came from Dr Ivan Sutherland, a Turing Award winner. He was taking a 15-months sabbatical in the EEE Department at Imperial College in the early 90s. He was designing an asynchronous multiplier at the time and wanted to optimized the sizes of transistors to make the circuit fast, and not just knowing how fast a given circuit runs. With this as a goal, he "invented" the method of Logical Effort.

The method of Logical Effort provides a simple method "*on the back of an envelope*" to choose the best topology and number of stages of logic for a function. Based on the linear delay model, it allows the designer to quickly estimate the best number of stages for a path, the minimum possible delay for the given topology, and the gate sizes that achieve this delay.

I learned about Logic Effort during the time it was created and subsequently taught it on my course for many years. David Harris learned about this while he was a PhD student at University of Stanford. He then took Ivan's notes and expanded on the materials with exercises and examples, and wrote the one-and-only "textbook" on Logic Effort published. in 1999.

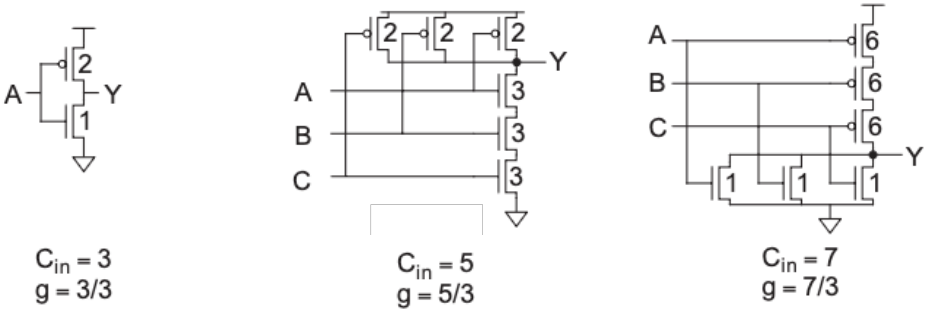
The timing analyzer tools do not use Logic Effort. However, Harris referred to Logic Effort throughout his textbook, I decided that I must include it in the module. Its use in industry is less popular nowadays.

What is Logical Effort?

- Logical effort = input capacitance of the gate / input capacitance of an inverter that can deliver the same output current.

Logical Effort: $g = C_{in_{gate}}/C_{in_{inv}}$

Effort delay f = logical effort g × electrical effort h



$C_{in} = 3$
$g = 3/3$

$C_{in} = 5$
$g = 5/3$

$C_{in} = 7$
$g = 7/3$

W&H p156

The formal definition of Logic Effort is shown in the slide. It is a measure, relative to an inverter, the "effort" needed to push the logic through this gate.

Consider the inverter on the left. The nMOS has a width of 1 (smallest). The pMOS, due to its lower mobility, has a width of 2. Therefore the inverter present its input with a capacitor load of 3 units (1C is gate capacitor for a unit nMOS transistor).

Logical effort of the inverter is defined to be 1.

Now consider the 2nd circuit – 3-input NAND gate. The pulldown series nMOS transistors have width of 3, so that when it is pulling down, the W/L ratio is still 1. The pulldown current capability of the NAND gate is the same as the of the inverter.

As to the pullup pMOS transistors, since they are in parallel, the width only has to be 2 for them to have the same pullup current as the inverter. Now this 3-input NAND gate has the same current drive capability as that of a inverter. However, the input capacitance is now 5 times that of a unit capacitance. Therefore the g = 5/3.

Working out for yourself why g for the 3-input NOR gate is 7/3.

Logical Effort of Common Gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		4/3	5/3	6/3	$(n+2)/3$
NOR		5/3	7/3	9/3	$(2n+1)/3$
tristate, multiplexer	2	2	2	2	2
XOR, XNOR		4, 4	6, 12, 6	8, 16, 16, 8	

- Logical effort increases with number of inputs.
- Logical effort for XOR/XNOR gates are different for different inputs.

W&H p156

This is a table showing the Logical Effort for common gate circuits with a variety of inputs.

XOR and XNOR is complex and the logical effort is dependent on both the circuit topology and which input is being considered.

Parasitic Delay of Common Gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		2	3	4	*n*
NOR		2	3	4	*n*
tristate, multiplexer	2	4	6	8	2*n*

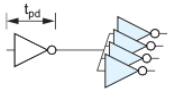
W&H p156

So far, we have only considered the effort delay using Logic Effort. The total delay also include the parasitic delay, which is a constant for a given gate and dependent on the fanout. Parasitic delay is the delay of a gate when its output is open circuit (no loading).

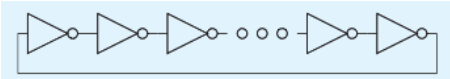
Here is a table showing the parasitic delays for common gates.

Estimate logic delay using Logical Effort

- Estimate the delay of a fanout-of-4 (FO4) inverter in L5S10.
  - Logical effort of inverter g = 1. Electrical effort h = 4.
  - Parasitic delay $p_{inv} \approx 1$.
  - Total delay $d = gh + p = 5$.



- A ring oscillator is constructed from an odd number of inverters. If a 31-stage ring oscillator has an output frequency of 2.7GHz, what is the unit delay of the inverter?



- Logical effort of inverter g = 1. Electrical effort h = 1.
- Parasitic delay $p_{inv} \approx 1$. Total delay for each inverter: $d = gh + p = 2$.
- N stage delay = 2N = half period of oscillation.
- $Period\ of\ oscillation = \frac{1}{2.7GHz} = 2 \times 2 \times 31 \times t_{pd_{inv}}$

$t_{pd_{inv}} = \tau = 3ps$

W&H p156

This slide shows two examples on using Logic Effort to estimate delays.

The first one is a simple two-stage inverter (e.g. clock circuit in a clock tree), with one inverter driving four identical inverters. (We called this a FO4 circuit.). The delay is easily derived to be 5 unit delay.

How can we find what is the unit delay? This is demonstrated in the second example.

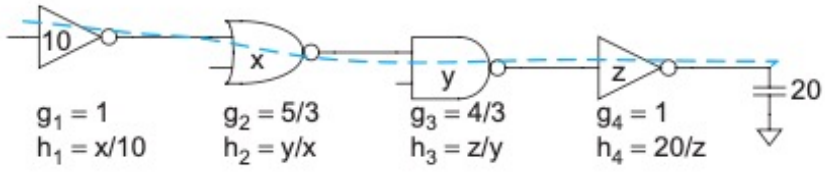
A ring oscillator is a chain ODD number of inverters feedback on itself. This circuit oscillates indefinitely. Given that the oscillation frequency is measured to be 2.7GHz, what is the value of the unit delay.

Using Logic Effort, we calculate the delay for each stage is 2, and for N stages, it is 2N. This is half the oscillation period. Therefore $t_{pd_{inv}}$ is calculated to be 3ps. Even for a process that is nearly 20 years old, we can achieve amazing speed in VLSI!

In VLSI, one usually add a ring oscillator to check the speed of the manufacture chip. However, the first inverter is replaced with a NAND gate so that the oscillator can be disabled. The input control and the output of the oscillator is connect to an external pins for measurement. It is common to design a ring oscillator to run at, say, 100MHz, so that the output waveform can be measured accurately.

Apply Logical Effort to Multistage Network

- One application of logic effort is to size the transistor in a multistage network to optimize its delay.
- Logical effort can help to determine best sizes for x, y and z for driving a capacitance load of 20.
- Define Path logical effort G as: $G = \prod g_i$
- Define Path electrical effort H as: $H = \frac{C_{out(path)}}{C_{in(path)}}$
- Then, the Path effort is: $F = \prod f_i = \prod g_i h_i$
- Note that $F \neq GH$, unlike the case of individual stage where $f = gh$



W&H p163

The above diagram show the logical and electrical efforts of each stage in a multistage path as a function of the sizes of each stage. The path of interest is marked with the dashed blue line. Observe that logical effort is **independent** of size, while electrical effort **depends** on sizes. We can develop some metrics for the path as a whole that are independent of sizing decisions.

The **path logical effort G** can be expressed as the products of the logical efforts of each stage along the path.

$$G = \prod g_i$$

The **path electrical effort H** is the ratio of the **output capacitance** the path must drive divided by the **input capacitance** presented by the path. This is more convenient than defining path electrical effort as the product of stage electrical efforts because we do not know the individual stage electrical efforts until gate sizes are selected.

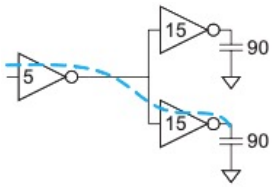
$$H = \frac{C_{\text{out(path)}}}{C_{\text{in(path)}}}$$

The path effort F is the product of the stage efforts of **each stage**.

$$F = \prod f_i = \prod g_i h_i$$

Branching Effort

- Only half the output current of the input inverter is used to drive the blue path. We therefore need something to account for this branching effect.
- **Branching effort b** of a branching node is: $b = \frac{C_{onpath} + C_{offpath}}{C_{onpath}}$.
- **Path branching effort B** is: $B = \prod b_i$
- Now we can define the **path effort F** as: $F = GBH$.



W&H p164

Recall that the stage effort of a single stage is $f = gh$.

Can we by analogy state $F = GH$ for a path? The answer is no. Generally $F \neq GH$.

The reason is that if there is a branching node as shown above, only half of the available current is used to drive the path of interest (the "onpath"). The other half of the current is driving the other path, the "offpath".

To account for this, we define a branching effort for a branching node as:

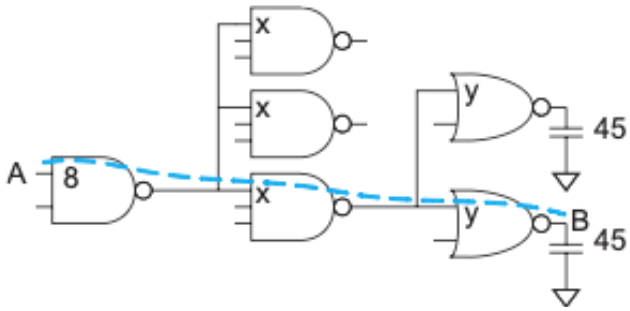
$$b = \frac{C_{\text{onpath}} + C_{\text{offpath}}}{C_{\text{onpath}}}$$

The total path branching effort B is $B = \prod b_i$

Example of using Logical Method (1)

Problem:

- Input NAND2 presents has capacitance of 8 transistor widths.
- Output load is equivalent to 45 transistor widths.
- Estimate the minimum delay of the path from A to B.
- Choose transistor sizes to achieve this delay.



W&H p165

This is the statement of the problem. You can find this in Example 4.13 on page 165 of W&H.

The goal is, given that we know input capacitance of NAND2 as 8 unit gate width, and that the load is 45 equivalent gate width, estimate the best delay for this circuit WITHOUT knowing the size of internal transistors.

Then find the transistor sizes so that the best delay is achieved.

We will solve this in two separate steps: 1) find the best delay; 2) work backward to find transistor sizes X and Y.

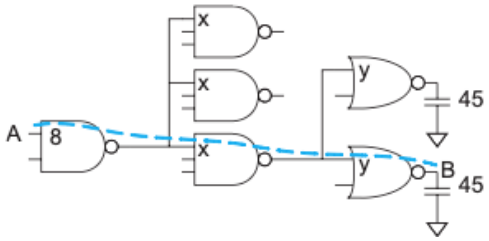
$$b = \frac{C_{\text{onpath}} + C_{\text{offpath}}}{C_{\text{onpath}}}$$

$$B = \prod b_i$$

Example of using Logical Method (2)

Solution to best delay calcu;ation:

- Path logical effort G = (4/3) x (5/3) x (5/3) = 100/27.
- Path electrical effort H = 45/8.
- Path branching effort B = 3 x 2 = 6.
- Total path effort F = GBH = 125.
- Best stage effort = divide effort equally in 3 stage $\hat{f} = \sqrt[3]{125} = 5$.
- Path parasitic delay P = 2+3+2 = 7.
- Therefore, minimum path delay is D = 3 x 5 + 7 = 22 in units of τ.
- $\tau \approx 3ps$ for 65nm. Therefore, delay is 66ps!



W&H p165

The solution here shows step-by-step the way the best delay is calculated using the method of Logical Method for the path in blue dotted line.

The key idea here is that we have three stages of logic. The best delay is achieved when the total path effort (normalised delay) is shared equally among the three stages.

Note that we managed to estimate the best delay for this circuit is 66ps WITHOUT knowing sizes of X and Y!

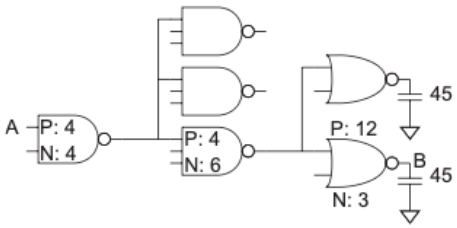
$$b = \frac{C_{\text{onpath}} + C_{\text{offpath}}}{C_{\text{onpath}}}$$

$$B = \prod b_i$$

Example of using Logical Method (3)

Solution to sizes of transistors:

- Stage effort $\hat{f} = 5$.
- We know that $\hat{f} = g \times h = g \times \frac{C_{out}}{C_{in}}$ for each stage.
- Now, work from output backward:
  - NOR2 $C_{out} = 45$, therefore $C_{in} = \frac{g \times C_{out}}{\hat{f}} = \frac{\frac{5}{3} \times 45}{5} = 15$.
  - Therefor the NOR2 gate has pMOS width of 12, and nMOS width of 3.
  - NAND3 $C_{out} = 15 + 15 = 30$, therefore $C_{in} = \frac{g \times C_{out}}{\hat{f}} = \frac{\frac{5}{3} \times 30}{5} = 10$.
  - Therefore the NAND3 gate has pMOS width of 4, and nMOS width of 6.



W&H p165

The solution to the sizes of transistors are shown in the slide. Once we know the stage effort $\hat{f} = 5$, we can work backward to find the total gate width at the input of each stage.

Example 4.13 on page 165 provide detail explanations. It also check that once all the widths are derived, it work forward from input to output and check that the delay is indeed 22τ.

$$b = \frac{C_{\text{onpath}} + C_{\text{offpath}}}{C_{\text{onpath}}}$$

$$B = \prod b_i$$

Choosing the Best Number of Stages

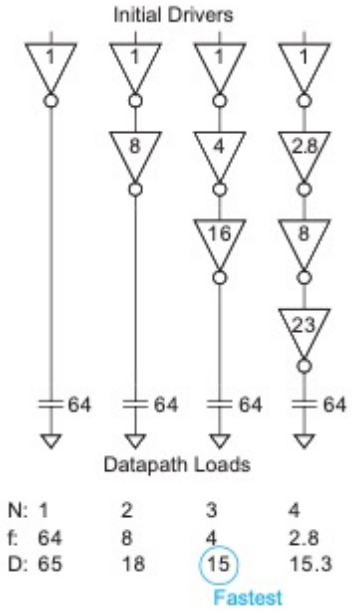
- For N-stage network with path effort F, minimum possible delay D is:

$$D = NF^{\frac{1}{N}} + P$$

where P is the path's parasitic delay.

- Key result from Logical Effort – no need to know sizes of transistors.
- How to choose N?
- Example of an inverter chain (see Example 4.14 in textbook).
- Path electrical effort $H = \frac{64}{1} = 64$.
- Path logical effort $G = 1$.
- Branching effort $B = 1$.
- Path effort $F = GBH = 65$.
- Best number of stage $N = \sqrt[N]{64}$.
- Delay $D = N\sqrt[N]{64} + N$

W&H p165-166



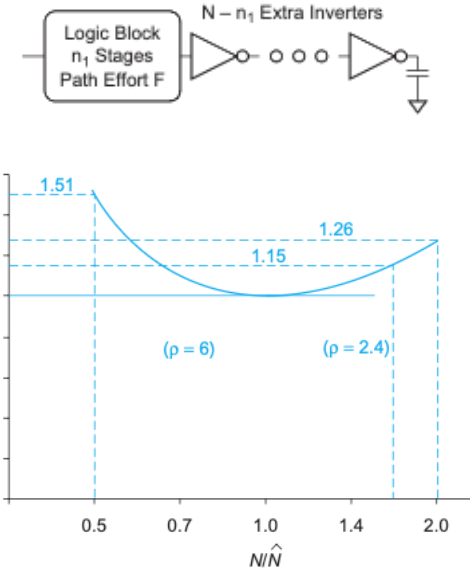
a multistage path with a path effort F, what is the optimial number of stages?

Adding inverter stages to optimize timing

- Best number of stages = N
- If N know, can always add inverters (buffers).
- How to find $\hat{N}$ for a given path effort F?
- Theoretically shown by Mead and Conway that:

  $\hat{N} = \log_e F$, where e = 2.718.
- Harris use simulation to show that, for chain of inverters, best number of stages is:

  $\hat{N} = \log_4 F$.
- The graph shows the sensitivity of delay to the number of stages



W&H p167

In general, you can always add inverters to the end of a path without changing its function (save possibly for polarity).

The logic block above has n_1 stages and a path effort of F. Consider adding N – n1 inverters to the end to bring the path to N stages. The extra inverters do not change the path logical effort but do add parasitic delay.

The Weste & Harris book provides derivation of the equations above. Given F, without knowing details of internal transistor sizes, one can estimate the best number of stages using the equation:

$$\hat{N} = \log_{3.59} F$$

Further, one can estimate the delay of this path with:

$$D = NF^{1/N} + \sum_{i=1}^{n_1} p_i + \left(N - n_1\right) p_{\text{inv}}$$

I recommend you reading the textbook p165 to 167 for more details.

Method of Logical Effect - Summary

1. Compute path effort. $F = GBH$
2. Estimate best number of stages. $N = \log_4 F$
3. Sketch path with N stages.
4. Estimate least delay. $D = NF^{\frac{1}{N}} + P$
5. Determine best stage effort. $\hat{f} = F^{\frac{1}{N}}$
6. Find gate sizes. $C_{in_i} = \frac{g_i C_{out_i}}{\hat{f}}$

W&H p170

This table summarizes all the equations associated with the method of Logical Effort.

TABLE 4.2 Logical effort of common gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		4/3	5/3	6/3	$(n+2)/3$
NOR		5/3	7/3	9/3	$(2n+1)/3$
tristate, multiplexer	2	2	2	2	2
XOR, XNOR		4, 4	6, 12, 6	8, 16, 16, 8	

TABLE 4.3 Parasitic delay of common gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		2	3	4	n
NOR		2	3	4	n
tristate, multiplexer	2	4	6	8	$2n$

Logical Effort Equations

Term	Stage Expression	Path Expression
number of stages	1	N
logical effort	g (see Table 4.2)	$G = \prod g_i$
electrical effort	$h = \frac{C_{out}}{C_{in}}$	$H = \frac{C_{out(path)}}{C_{in(path)}}$
branching effort	$b = \frac{C_{onpath} + C_{offpath}}{C_{onpath}}$	$B = \prod b_i$
effort	$f = gh$	$F = GBH$
effort delay	f	$D_F = \sum f_i$
parasitic delay	p (see Table 4.3)	$P = \sum p_i$
delay	$d = f + p$	$D = \sum d_i = D_F + P$

W&H p170

This table summarizes all the equations associated with the method of Logical Effort.

TABLE 4.2 Logical effort of common gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		4/3	5/3	6/3	$(n+2)/3$
NOR		5/3	7/3	9/3	$(2n+1)/3$
tristate, multiplexer	2	2	2	2	2
XOR, XNOR		4, 4	6, 12, 6	8, 16, 16, 8	

TABLE 4.3 Parasitic delay of common gates

Gate Type	Number of Inputs				
	1	2	3	4	*n*
inverter	1				
NAND		2	3	4	n
NOR		2	3	4	n
tristate, multiplexer	2	4	6	8	$2n$

Limitation of Logical Effort

- Chicken and egg problem.
  - Need path to compute G.
  - But don't know number of stages without G.
- Simplistic delay model.
  - Neglects input rise time effects.
- Interconnect is not taken into account.
  - Iteration required in designs with wire.
  - Best suited to high-speed regular array with little interconnect wires.
- Maximum speed only.
  - Not minimum area/power for constrained delay.

W&H p170

Logical Effort does not account for interconnect.

Logical Effort is most applicable to high-speed circuits with regular layouts where routing delay does not dominate. Such structures include adders, multipliers, memories, and other data- paths and arrays.

Logical Effort explains how to design a critical path for maximum speed, but not how to design an entire circuit for minimum area or power given a fixed speed constraint.

Paths with nonuniform branching or reconvergent fanout are difficult to analyze by hand.

The linear delay model fails to capture the effect of input slope. Fortunately, edge rates tend to be about equal in well-designed circuits with equal effort delay per stage.

These limitations mean that Logical Effort is only used for designing small, highly optimized subcircuits, and not used in general CAD software.

Timing Analyzer Delay Models

- Slope-based linear model:
  - Linear delay model but include effect of input signal slope.
  - Suffers limitations of linear models, therefore not generally used.
- Nonlinear Delay Model:
  - Table storing delay vs load capacitances, input slope etc.
  - Use table lookup and interpolation.
  - Cannot handle complex RC network or noise events.
- Current Source Model:
  - Models output current as non-linear current source.
  - Liberty Composite Current Source model (CCSM) (widely used) - stores output current as function of input slew rate and output capacitance.

W&H p173-174

Slope-based Linear Model:

A simple approach, although now out dated, is to extend the linear delay model by adding a term reflecting the input slope and assume that the slope of the input is proportional to the delay of the previous stage. Linear delay models are not accurate enough to handle the wide range of slopes and loads found in synthesized circuits, so they have largely been superseded by nonlinear delay models.

Nonlinear Delay Model:

A nonlinear delay model looks up the delay from a table based on the load capacitance and the input slope. Separate tables are used to lookup rising and falling delays and output slopes. Nonlinear delay models do not contain enough information to characterize the delay of a gate driving a complex RC interconnect network with sufficient accuracy. It is also now consider out dated.

Current Source Model:

A current source model expresses the output current as a nonlinear function of the input and output voltages of the cell. A timing analyzer numerically integrates the output current to find the voltage as a function of time into an arbitrary RC network and to solve for the propagation delay.

The ***Liberty Composite Current Source Model*** (CCSM) stores output current as a function of time for a given input slew rate and output capacitance.

Further Reading

- Chapter 4 of Weste & Harris – detail explanations on Logical Method.

Slope-based Linear Model:

A simple approach, although now out dated, is to extend the linear delay model by adding a term reflecting the input slope and assume that the slope of the input is proportional to the delay of the previous stage. Linear delay models are not accurate enough to handle the wide range of slopes and loads found in synthesized circuits, so they have largely been superseded by nonlinear delay models.

Nonlinear Delay Model:

A nonlinear delay model looks up the delay from a table based on the load capacitance and the input slope. Separate tables are used to lookup rising and falling delays and output slopes. Nonlinear delay models do not contain enough information to characterize the delay of a gate driving a complex RC interconnect network with sufficient accuracy. It is also now consider out dated.

Current Source Model:

A current source model expresses the output current as a nonlinear function of the input and output voltages of the cell. A timing analyzer numerically integrates the output current to find the voltage as a function of time into an arbitrary RC network and to solve for the propagation delay.

The ***Liberty Composite Current Source Model*** (CCSM) stores output current as a function of time for a given input slew rate and output capacitance.